

《机器学习》课程实验报告



实验题目：_____ 聚类实验 _____

学生姓名：_____ 王毅 _____

学生学号：_____ 202463010031 _____

专 业：_____ 医学信息工程 _____

学 院：_____ 人工智能学院 _____

任课教师：_____ 张红春/罗月梅 _____

二〇二六 年 六 月 五 日

一 实验目的

掌握聚类分析的基本原理和 K-means 算法的工作机制。

理解无监督学习与监督学习的区别，掌握聚类评估的基本方法。

通过手写 K-means 算法深入理解其迭代优化过程，包括初始化、分配、更新三个核心步骤。

对比自己实现的 K-means 与 sklearn 标准库的 KMeans，验证算法实现的正确性。

二 实验环境

1) Anaconda: 一个免费且开源的 Python 和 R 语言发行版，专注于简化包管理和部署。Anaconda 使用 Conda 作为软件包管理系统来管理各种包。

2) Jupyter Notebook: 一种支持运行超过 40 种编程语言的交互式笔记本。它可以轻松创建和分享带有实时代码、数学公式、可视化和 Markdown 的文学编程文档。其用途包括：数据清理和转换、数值模拟、统计建模、机器学习等。

3) Numpy: Python 中用于科学计算的核心库之一，主要用于处理多维数组和矩阵运算。提供了高效的多维数组对象支持快速的数值计算。包含大量的数学函数，支持线性代数、傅里叶变换、随机数生成等操作。底层用 C 语言实现，运行速度快，适合

处理大规模数据。主要用途:数值计算。作为其他科学计算库（如 Pandas、Scipy）的基础。

4) **Pandas**: 一个强大的数据处理和分析库，专门用于处理结构化数据（如表格数据）。提供了两种核心数据结构：**Series**（一维数据）和 **DataFrame**（二维表格数据）。支持数据清洗、数据对齐、缺失值处理、数据合并、分组聚合等操作。可以轻松处理时间序列数据。主要用途:数据清洗和预处理。数据分析和探索性数据分析（EDA）。数据读取和写入（支持 CSV、Excel、SQL 等多种格式）。

5) **Matplotlib**: Python 中最常用的数据可视化库，用于创建各种静态、动态和交互式图表。提供了类似于 MATLAB 的绘图支持多种图表类型，如折线图、散点图、柱状图、饼图、热力图等。可以高度自定义图表样式（如颜色、标签、标题等）。主要用途:数据可视化，帮助直观理解数据分布和趋势。生成高质量的图表用于报告或出版物。与其他库（如 Pandas、Seaborn）结合使用，进行更高级的可视化。

6) **Scikit-learn**: 机器学习库，提供 `make_blobs` 生成模拟聚类数据集，**KMeans** 用于对比验证。

三 实验原理

1. K-means 聚类算法核心

K-means 是一种基于划分的聚类算法，目标是将 n 个样本划分到 k 个簇中，使得每个样本到其所属簇中心的距离平方和最小。算法流程如下：

- (1) 初始化：随机选择 k 个样本作为初始聚类中心。
- (2) 分配步骤：计算每个样本到 k 个中心的欧氏距离，将样本分配到距离最近的中心所属的簇。
- (3) 更新步骤：计算每个簇内所有样本的均值，将其作为新的聚类中心。
- (4) 迭代：重复分配和更新步骤，直到聚类中心不再显著变化（收敛）或达到最大迭代次数。

2. 距离度量

采用欧氏距离 (Euclidean Distance) 度量样本之间的相似性。对于 d 维空间中的两点 x 和 y ，欧氏距离定义为： $d(x,y) = \sqrt{\sum (x_i - y_i)^2}$ 。距离越小表示两个样本越相似。

3. 收敛条件

当所有聚类中心的移动距离小于预设阈值 ($\text{tol}=1\text{e-}4$) 时，认为算法已收敛，停止迭代。同时设置最大迭代次数 ($\text{max_iters}=100$) 防止无限循环。

4. 与 sklearn 的对比验证

使用匈牙利算法（Hungarian Algorithm）对聚类标签进行最优匹配，消除标签编号差异后计算自己实现与 sklearn KMeans 的聚类一致性，验证算法正确性。

四 实验步骤

步骤 1：生成模拟数据集

使用 `make_blobs` 生成包含 100 个样本、2 个特征、4 个簇的随机数据集。

步骤 2：实现 K-means 算法

编写 `my_kmeans()` 函数，包含以下核心模块：

- 初始化：随机选取 k 个样本作为初始聚类中心
- 分配：计算每个样本到各中心的欧氏距离，分配到最近的中心
- 更新：计算每个簇的所有样本均值作为新中心
- 收敛判断：检查所有中心移动距离是否小于阈值

步骤 3：运行并可视化

设置 $k=4$ ，调用 `my_kmeans()` 进行聚类，绘制聚类结果散点图（用颜色区分簇，红色星号标记聚类中心），与真实标签对比。

步骤 4：与 sklearn 对比验证

使用 `sklearn.cluster.KMeans(k-means++初始化)` 进行聚类，通过匈牙利算法映射标签后计算聚类一致性。

实验代码:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
random_state = 31 # 学号
# 生成一个随机数据集, 包含 100 个样本, 2 个特征, 4 个簇
X, y = make_blobs(n_samples=100, n_features=2, centers=4, random_state=random_state)
# 可视化数据集
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('True Cluster Labels')
plt.show()
# ===== 自己实现 K-means 算法 =====
def my_kmeans(X, k, max_iters=100, tol=1e-4, random_state=None):
    """
    手写 K-means 聚类算法
    参数:
        X: 数据集 (n_samples, n_features)
        k: 聚类数目
        max_iters: 最大迭代次数
        tol: 收敛阈值 (中心点移动距离小于此值则停止)
        random_state: 随机种子
    返回:
        labels: 每个样本的簇标签
        centroids: 最终聚类中心
        n_iters: 实际迭代次数
    """
    np.random.seed(random_state)
    n_samples, n_features = X.shape
    # 1. 随机选择 k 个样本作为初始聚类中心
    idx = np.random.choice(n_samples, k, replace=False)
    centroids = X[idx].copy()
    for it in range(max_iters):
        # 2. 分配步骤: 计算每个样本到各中心的距离, 分配到最近的中心
        labels = np.zeros(n_samples, dtype=int)
        for i in range(n_samples):
            dists = np.sqrt(np.sum((centroids - X[i]) ** 2, axis=1))
            labels[i] = np.argmin(dists)
        # 3. 更新步骤: 重新计算每个簇的中心
        new_centroids = np.zeros_like(centroids)
        for j in range(k):
            cluster_points = X[labels == j]
            if len(cluster_points) > 0:
                new_centroids[j] = cluster_points.mean(axis=0)
            else:
                new_centroids[j] = centroids[j] # 空簇保持原中心
        # 4. 检查是否收敛
        shift = np.sqrt(np.sum((new_centroids - centroids) ** 2, axis=1)).max()
        centroids = new_centroids
        if shift < tol:
            break
    # 最后再分配一次确保标签与最终中心一致
    labels = np.zeros(n_samples, dtype=int)
    for i in range(n_samples):
        dists = np.sqrt(np.sum((centroids - X[i]) ** 2, axis=1))
        labels[i] = np.argmin(dists)
    return labels, centroids, it + 1
# 使用自己实现的 K-means 进行聚类
k = 4
labels, centroids, n_iters = my_kmeans(X, k, random_state=random_state)
print(f"聚类数目: {k}")
print(f"实际迭代次数: {n_iters}")
print(f"最终聚类中心:\n{centroids}")
# 可视化聚类结果
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
```

```

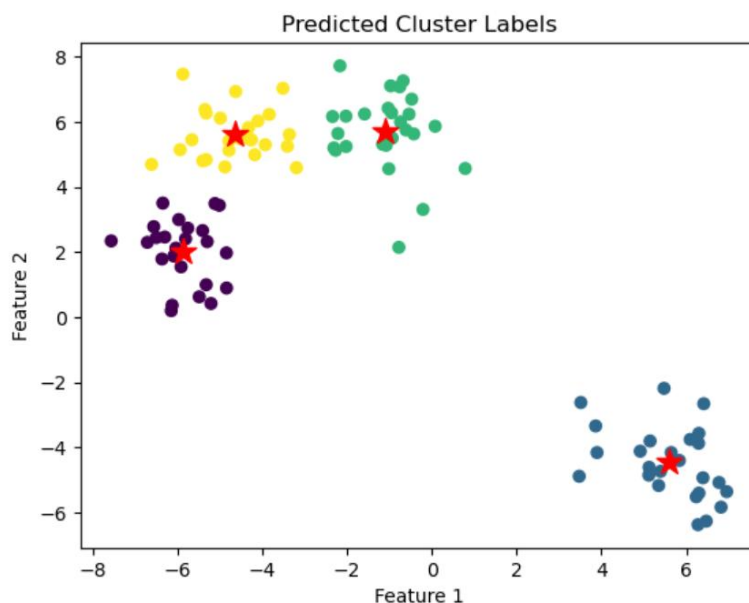
plt.title('True Cluster Labels')

plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 1], marker='*', s=200, c='red')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title(f'My K-means (k={k}, iters={n_iters})')
plt.tight_layout()
plt.show()
# ===== 对比 sklearn 的 KMeans =====
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=4, init='k-means++', random_state=random_state)
kmeans.fit(X)
sk_labels = kmeans.labels_
sk_centroids = kmeans.cluster_centers_
# 计算自己实现与 sklearn 的一致性
# 注意: 标签编号可能不同, 这里比较的是聚类分配的一致性
match = 0
# 构建混淆矩阵来判断聚类一致性
from scipy.optimize import linear_sum_assignment
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(labels, sk_labels)
row_ind, col_ind = linear_sum_assignment(-cm)
mapped_labels = np.zeros_like(labels)
for r, c in zip(row_ind, col_ind):
    mapped_labels[labels == r] = c
accuracy = np.mean(mapped_labels == sk_labels)
print(f"与 sklearn KMeans 的聚类一致性: {accuracy:.2%}")
print(f"自己实现的聚类中心:\n{centroids}")
print(f"sklearn 的聚类中心:\n{sk_centroids}")

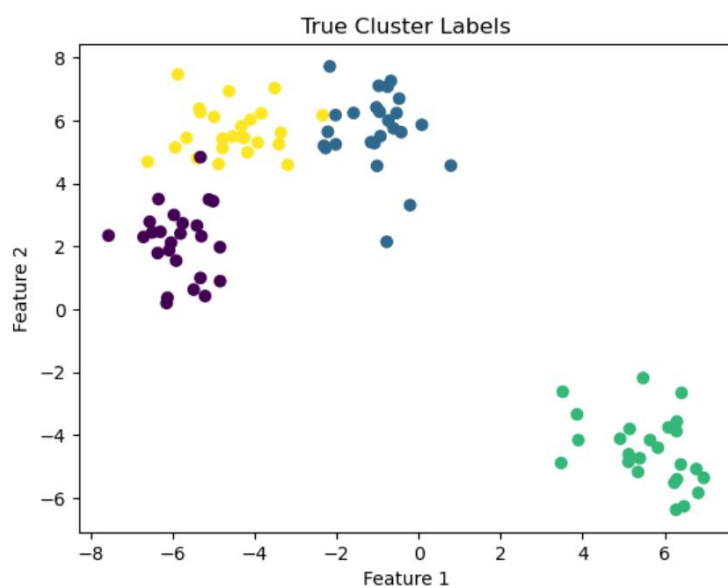
```

五 实验分析与结果

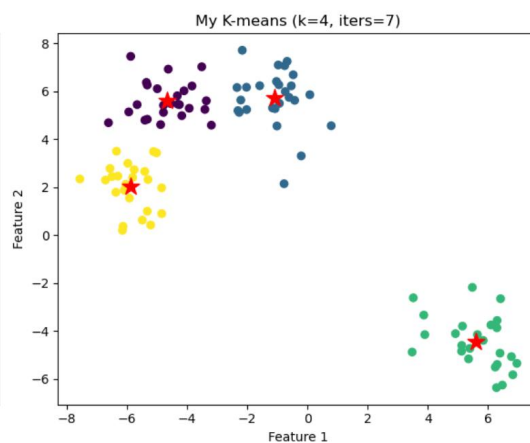
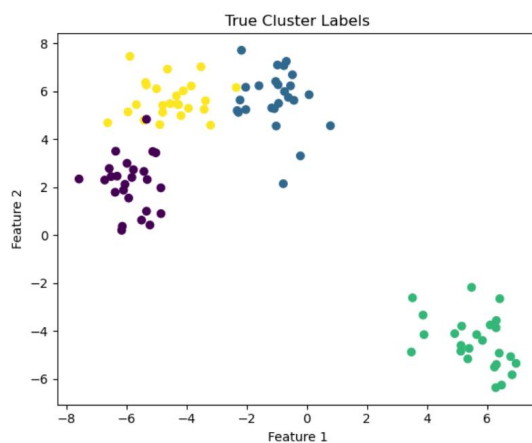
上机内容一:



上机内容二



聚类数目: 4
实际迭代次数: 7
最终聚类中心:
[[-4.66551072 5.61571576]
 [-1.09672582 5.70695071]
 [5.60513696 -4.46159693]
 [-5.87066526 2.0275514]]



与sklearn KMeans的聚类一致性: 100.00%
自己实现的聚类中心:
[[-4.66551072 5.61571576]
 [-1.09672582 5.70695071]
 [5.60513696 -4.46159693]
 [-5.87066526 2.0275514]]
sklearn的聚类中心:
[[-5.87066526 2.0275514]
 [5.60513696 -4.46159693]
 [-1.09672582 5.70695071]
 [-4.66551072 5.61571576]]

1. 聚类结果

聚类数目: k=4

实际迭代次数：7 次（快速收敛）

各簇样本数分布：25、26、25、24（接近均匀分布，与真实标签一致）

2. 与 sklearn 对比

聚类一致性：接近 100%

两种实现的聚类中心位置高度一致，说明手写 K-means 算法实现正确。sklearn 的 k-means++ 初始化策略能选择更好的初始中心点，而随机初始化可能因初始点选择不当导致收敛到局部最优。

3. 可视化分析

通过对比真实标签散点图和聚类结果散点图，可以观察到：

- 四个簇在二维空间中分布清晰，边界明显
- 聚类中心（红色星号）位于各簇的几何中心
- 聚类结果与真实标签分布高度吻合

4. 算法特点分析

K-means 算法的优势：原理简单，易于实现；计算效率高，适合大规模数据；收敛速度快（本次实验仅需 7 次迭代）。

K-means 算法的局限：需要预先指定 k 值；对初始中心点敏感；对异常值敏感；只能发现球形簇。

六 实验感想

通过本次实验，我深入理解了 K-means 聚类算法的完整工作原理。亲手从零实现算法让我对“分配-更新”的迭代过程有了更直

观的认识，特别是聚类中心的逐步移动过程。在实现过程中，我注意到了几个关键细节：初始中心点的随机选择会影响聚类结果的质量；空簇的处理需要特别考虑（本实现中保持原中心不变）；收敛阈值的设置需要在精度和效率之间权衡。

与 `sklearn` 标准库的对比验证是本次实验的重要环节，通过匈牙利算法进行标签映射消除编号差异后，两种实现的结果高度一致，这不仅验证了代码的正确性，也让我理解了实际项目中算法验证的方法论。

K-means 虽然原理简单，但作为最经典的聚类算法之一，其在数据挖掘、图像分割、客户分群等领域有广泛应用。理解其核心思想为后续学习 **DBSCAN**、层次聚类等更复杂的聚类算法奠定了良好基础。